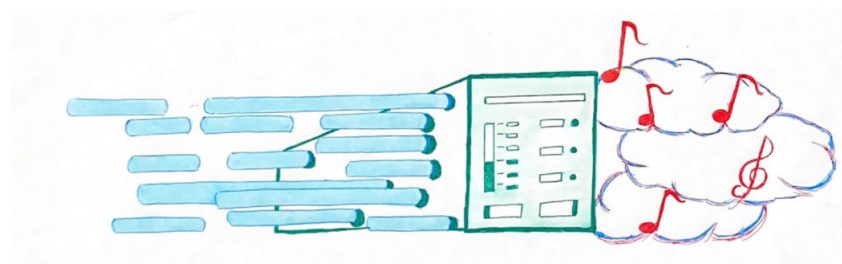


An Overview of the Algorithms Behind the Internet's Most Popular Audio Codec

Victor Veytsman



1 Introduction

This article looks at the algorithms behind the Opus codec, a codec popular because of its versatility and its high performance. We will touch on how analog audio is converted into digital signals, and then how these signals are compressed in various ways to make them suitable for applications such as live audio and video chat and distributing compressed music.

2 What is a Codec?

A codec is an encoder and decoder of a data stream. First, in the hardware world, codecs are devices that convert the analog signal of audio into a digital signal. Then, in the software world, codecs compress the raw data so that it can be stored and transmitted more efficiently. Some software codecs are lossless, meaning they compress the audio without losing any of the raw data, but most are lossy, meaning they lose some of the raw data when compressing it. The challenge in creating a lossy codec is to use as little space as possible while still having high-quality audio.

3 Turning Waves into Bits

The first step in transmitting audio through a computer or storing it in the computer's memory is to turn it into ones and zeros. This is done with **Pulse Code Modulation (PCM)**. Pulse Code Modulation involves three main steps: **Sampling**, **Quantization**, and **Encoding**.

3.1 Sampling

The first step in Pulse Code Modulation is Sampling. When the device receives a wave, it takes samples at regular intervals. Each sample measures the amplitude of the wave.

The **sampling rate** measures how often samples are taken. This number can vary, but usually falls between 40 kHz and 50 kHz (thousand times per second). For example, CD audio is sampled at 44.1 kHz. It is very common for digital equipment to sample at 48 kHz for a variety of reasons that mostly boils down to its ability to deal with the range of human hearing and its many nuances. (This is important because when playing a video, it is convenient when the number of samples played between frames is a whole number.)

3.2 Quantization

The next step is Quantization. In this step, each sample is fit to a range of predefined values, essentially rounding the sample. There are a few algorithms that can select quantization values that are optimal, in the sense that less rounding needs to be done overall. For example, some audio waves are more often close to zero than they are to 1 or -1, so algorithms might use more quantization values in low ranges than in higher ones. This is true of algorithms such as the Mu-law and the A-law algorithms, which are algorithms used to sample audio for digital telecommunication. (Mu-law is used in the United States and Japan, and A-law is used in Europe.) However, it is just as common to choose quantization values that are equidistant from each other. PCM that uses these quantization values is also called Linear Pulse Code Modulation (LPCM).

The **bit depth** is the number of bits each sample uses. A bit is a single one or zero.

3.3 Encoding

Encoding is the final step of Pulse Code Modulation. Exact methods of encoding can differ, but for LPCM it can be as simple as specifying the sample rate and the bit depth at the beginning of the file, and then writing in the data from each sample.

The **bit rate** is a function of the bit depth and the sampling rate, and describes how much data is required to encode a certain length of audio. It is important to know the bit rate because when transmitting audio over the Internet in live instances, one needs to make sure that the bit rate isn't so high that the audio can't keep up with the rate at which the network can transmit data (the bandwidth).

4 What is Opus?

Opus is a codec that has been around since 2010 but is based on earlier technologies in the field. It is a codec that does lossy compression, meaning that while compressing the audio it loses

some of the data. It has found widespread usage in applications like Zoom and Discord because of its versatility and its ability to produce great sound in many conditions.

Opus supports sampling rates between 8 and 48 kHz and bit rates between 6 and 510 kB/s (1 kB/s = 8000 ones and zeros per second). It can also seamlessly switch between different sampling rates and bit rates, making it perfect for transmitting over a network that might not be the most stable. It backs this strength up by being fast enough that it can be used for live communication with little delay. Some programs, such as the program “Jamulus,” use the codec to allow musicians to play with each other seamlessly over the Internet, despite the fact that musicians can notice a delay of as little as 30 milliseconds.

Opus manages this versatility by operating in three different modes, switching between the modes seamlessly as conditions change:

- The SILK mode, used for speech at lower bit rates
- The CELT mode, used for music and other high-quality recording
- The hybrid mode, in which SILK is used for lower frequencies (up to 8 kHz) while CELT is used for higher frequencies (up to 20 kHz)

5 SILK and Linear Predictive Coding

The SILK mode is based off of the SILK codec, which was first created by Skype for speech. The algorithm behind the SILK codec is **Linear Predictive Coding**, which is also used for speech synthesis.

In Linear Predictive Coding, the audio is encoded in chunks, between 2.5 and 20 milliseconds long. Each chunk includes the first few samples, a prediction coefficient, and a set of errors. The decoder then reconstructs each subsequent sample using the coefficient and the previous samples it has processed. The errors themselves are smaller to transmit than the samples, and their size diminishes as the prediction coefficient becomes more accurate.

When decoding the audio, the decoder uses the following equation:

$$X_t = \sum_{k=1}^p a_k X_{t-k} + e_t$$

X_t is the calculated sample, p is the number of samples in the chunk, and e_t is the error for that sample. Most importantly, a_k is the prediction coefficient mentioned earlier. There are several different algorithms that can be used to find prediction coefficients. Opus uses **Burg’s Method**, a method originally proposed in the field of geophysics and was first explored for the use of audio

in the 1980s. On top of that, Opus includes multiple methods that further increase the accuracy of the coefficients, including methods that analyze pitch and vocal activity.

6 CELT and Modified Discrete Cosine Transform

The other major piece of the Opus puzzle is its CELT mode, which is based off of Xiph.Org Foundation's open codec CELT codec, which would later contribute to the creation of Opus. The main algorithm behind the CELT mode is **Modified Discrete Cosine Transform**.

A **transform** turns a function (such as an audio waveform) into a weighed sum of waves, such that the original form can be calculated by summing each of the transformed waves, each multiplied by their coefficient (their weight). You may have heard of this in the context of Fourier Transforms. In a Fourier Transform, the waves are complex sinusoids, but in Discrete Cosine Transforms, each wave is a cosine function.

When most of the weights are close to zero, the input is easier to store because you can skip storing those weights. This is helpful in the compression of audio and images, and is the reason Discrete Cosine Transforms are used in JPEGs, MP3s, and other file types.

The Modified Discrete Transform Algorithm looks like this:

$$X_k = \sum_{n=0}^{2N-1} x_n \cos \left(\frac{\pi}{N} \left(n + \frac{1}{2} + \frac{N}{2} \right) \left(k + \frac{1}{2} \right) \right)$$

In this equation, X_k is the output frame, x_n is the input, and N is the number of frames in total. What separates this transform from others is that it takes $2N$ data points and produces N outputs, as can be seen in the sum. This is because each input to the transform overlaps with its neighbors, which is helpful in eliminating any artifacts (distortions caused by compression) that come at the beginning and end of each frame.

Opus uses this algorithm in frames, much like the chunks of the SILK model discussed earlier. These frames can be between 2.5 and 20 milliseconds. Once again, Opus includes methods to make the compression more efficient and accurate, including a pre-filter and post-filter. One of their important functions is to separate the gain (the energy, or the volume) from the shape of the sound, to make it easier to transport and restore on the other side. Often, the codec will also opt to use multiple small MDCTs rather than one for the entire frame, in case there is a large burst of energy (a **transient**) in a frame.

7 Conclusion

This article provides a brief overview of the main algorithms that power the Opus codec. In reality, the codec includes much more than what is discussed here. Essentially, the codec is a culmination of decades of research and implementation done by dozens, if not hundreds, of scientists. Its complexity makes it the best codec available for almost all use cases, and with such meticulously thought-out features, it is no wonder that the codec is so widespread across the Internet today.

References

- Appleton, Nick. 2013. “Understanding the Modified Discrete Cosine Transform (MDCT),” September 15, 2013. <https://www.appletonaudio.com/blog/2013/understanding-the-modified-discrete-cosine-transform-mdct/>.
- Berry, Gus, and Peter Rodocker. n.d. “Understanding audio bitrate.” <https://www.adobe.com/creativecloud/video/discover/audio-bitrate.html>.
- Gray, A., and D. Wong. 1980. “The Burg algorithm for LPC speech analysis/Synthesis.” *IEEE Transactions on Acoustics, Speech, and Signal Processing* 28 (6): 609–615. <https://doi.org/10.1109/TASSP.1980.1163489>.
- Hollerweger, Florian. 2012. “Opus, the revolutionary open audio codec for podcasts and internet audio,” September 26, 2012. <https://auphonic.com/blog/2012/09/26/opus-revolutionary-open-audio-codec-podcasts-and-internet-audio/>.
- Ivannikov, Georgii. 2019. “1. Linear Predictive Coding,” September 12, 2019. <https://www.youtube.com/watch?v=i8F2RzDPTLc>.
- “Pulse Code Modulation and Demodulation.” n.d. <https://www.elprocus.com/pulse-code-modulation-and-demodulation/>.
- Sapp, Craig. n.d. “WAVE PCM soundfile format.” <http://soundfile.sapp.org/doc/WaveFormat/>.
- Valin, Jean-Marc. 2021. “Opus, the Swiss Army Knife of Audio Codecs,” January 20, 2021. <https://www.youtube.com/watch?v=iaAD71h9gDU>.
- Valin, JM., Mozilla Corporation, K. Vos, Skybe Technologies S.A., and T Terriberry. 2012. “Definition of the Opus Audio Codec,” September. <https://datatracker.ietf.org/doc/html/rfc6716#section-4.3>.
- Veen, Barry Van. 2020. “The Discrete Cosine Transform,” March 19, 2020. <https://www.youtube.com/watch?v=Aq7YgjbVrPg>.